

“초등부 1번. 보물 찾기” 문제 풀이

작성자: 김준원

부분문제 4

다빈은 위치 S 를 조사하고, 이로부터 1칸 오른쪽 위치를 조사하고, 이로부터 2칸 왼쪽 위치를 조사하고, 이로부터 3칸 오른쪽 위치를 조사하고, ...를 진행한다. 처음으로 L 또는 R 과 같은 위치를 만나면 놀이를 끝낸다. 이를 그대로 반복문을 이용해 코드로 옮기면 부분문제 1, 2, 3, 4를 풀 수 있다.

다음은 관찰하면 동일한 풀이를 조금 더 쉽게 구현할 수 있다. 다빈이 조사하는 위치는 차례대로 $S, S+1, S-2, S+3, S-4, \dots$ 이다. 즉, x 번째 단계에서 조사할 위치는 처음 조사하는 위치 S 로부터 $x-1$ 만큼 떨어져 있으며, 떨어져 있는 방향은 x 의 홀짝성에 따라 달라진다. 즉, x 번째 단계에서 조사하는 위치는:

- x 가 짝수 ($x = 2k$)이면, $S + k$
- x 가 홀수 ($x = 2k + 1$)이면, $S - k$

이다. 이를 반복문으로 구현하여, L 또는 R 과 같은 위치가 등장하였을 때의 단계를 출력하면 문제를 풀 수 있다. 경우의 수마다 정답의 크기에 비례하는 시간이 소요되므로, 정답의 크기가 대략 2,000보다 작은 부분문제 1, 2, 3, 4를 시간 안에 풀 수 있다.

부분문제 5

위의 관찰을 하면, x 번째 단계에서 보물을 찾는 경우는 항상 아래 두 가지 중 하나임을 알 수 있다.

- x 가 짝수 ($x = 2k$)일 때, 위치 $S + k$ 을 조사하여, 위치 L 에 있는 보물을 발견한다.
이 때, $S + k = L$ 이며, $k = L - S$, $x = 2 \times (L - S)$ 이다.
- x 가 홀수 ($x = 2k + 1$)일 때, 위치 $S - k$ 을 조사하여, 위치 R 에 있는 보물을 발견한다.
이 때, $S - k = R$ 이며, $k = S - R$, $x = 2 \times (S - R) + 1$ 이다.

따라서, $2 \times (L - S)$ 번째 단계와 $2 \times (S - R) + 1$ 번째 단계 중 먼저 도달하는 단계에서 보물을 찾을 수 있다.

따라서 정답은 수식 $\min(2 \times (L - S), 2 \times (S - R) + 1)$ 로 나타낼 수 있으며, 경우의 수마다 상수 시간에 문제를 해결할 수 있다.

“초등부 2번 / 중등부 1번. 가로등” 문제 풀이

작성자: 윤교준

부분문제 1

$x = 0$ 부터 $x = A_1$ 까지 정수 위치의 어두운 정도는 $0, 1, \dots, A_1$ 이다. 또한, $x = A_1 + 1$ 부터 $x = L$ 까지 정수 위치의 어두운 정도는 $1, 2, \dots, L - A_1$ 이다.

즉, 정렬된 어두운 정도 값에 간단한 규칙이 있으므로, A_1 과 $L - A_1$ 의 대소 비교 후 답을 출력할 수 있다.

부분문제 2

한 위치의 어두운 정도는 지문의 주어진 식에 따라 $\mathcal{O}(N)$ 에 계산할 수 있다.

따라서, 모든 위치의 어두운 정도를 계산한 후 정렬하면 $\mathcal{O}(NL)$ 의 시간 복잡도에 해결할 수 있다.

부분문제 3

가로등이 0부터 L 까지 등간격으로 떨어져 있다.

따라서, 출력한 수가 K 개가 될 때까지, 0을 N 번, 1 이상의 정수를 각각 $2(N - 1)$ 번씩 차례대로 출력하면 된다.

부분문제 4

다음을 관찰한다:

- 위치 $0 \leq x \leq A_1$ 의 어두운 정도는 $A_1 - x$.
- 위치 $A_i \leq x \leq A_{i+1}$ 의 어두운 정도는 $\min\{x - A_i, A_{i+1} - x\}$.
- 위치 $A_N \leq x \leq L$ 의 어두운 정도는 $x - A_N$.

각 구간에 대해, 각 위치의 어두운 정도는 $\mathcal{O}(1)$ 에 계산할 수 있다. 따라서, 모든 위치의 어두운 정도를 계산하여 정렬까지 $\mathcal{O}(L)$ 의 시간 복잡도에 수행할 수 있다.

부분문제 5

N 개의 가로등이 놓인 위치를 모두 출발지로 하여 BFS를 수행한다.

큐에서 (위치, 거리) 쌍을 제거할 때마다 거리 값을 출력하면, 그것이 문제에서 원하는 답이 된다.

큐에 삽입과 삭제는 총 $\mathcal{O}(N + K)$ 번 수행된다.

큐에 점을 삽입하기 전에 방문 여부를 검사해야 하며, 이는 `std::set` 등의 BBST 라이브러리를 사용하여 효율적으로 처리할 수 있다.

따라서, 총 시간 복잡도는 $\mathcal{O}((N + K) \lg(N + K))$ 이다.

$\mathcal{O}(N + K)$ 의 선형 시간과 공간 복잡도로도 문제를 해결할 수 있다.

“초등부 3번 / 중등부 2번. 색깔 모으기” 문제 풀이

작성자: 박찬솔

부분문제 1

$N \leq 2$ 이므로 가능한 모든 입력에 대해 답을 미리 구해두어 출력하면 된다.

부분문제 2

$N \leq 20$ 이므로 $dp[\text{bit}]$ 를 상태가 bit 일 때 초기 상태로부터의 최소 이동 횟수로 정의하면 비트마스킹을 사용한 동적 계획법으로 문제를 해결할 수 있다.

bit 의 i 번째 비트는 색깔이 i 인 공이 같은 상자에 있으면 1, 그렇지 않으면 0으로 둔다. 각 상태에서 나올 수 있는 현재 공의 상자 배치가 유일하므로 동적 계획법을 사용할 수 있다.

부분문제 5

i 번째 상자에서 아래에 있는 공의 색을 A_i , 위에 있는 공의 색을 B_i 라고 하자. 정점이 N 개인 방향 그래프 G 와 무방향 그래프 G' 을 고려하자. 모든 A_i, B_i 에 대해서 G 에 방향 간선 (A_i, B_i) 를 추가하고, G' 에 무방향 간선 (A_i, B_i) 를 추가한다. 만약 $A_i = B_i$ 이면 두 그래프에 자기 자신으로 가는 간선을 하나씩 추가한다.

그래프 G' 에서 모든 정점의 차수는 정확히 2 이므로 그래프는 단순 사이클의 집합으로 구성되어 있다. 서로 다른 단순 사이클에 속하는 정점은 영향을 주지 않으므로, 그래프에 있는 단순 사이클을 하나씩 처리하면 된다.

이렇게 만든 그래프 G' 의 각 사이클에 G 에서의 진출 차수가 2 이상인 정점이 1개 이하씩 있다면 아래 과정을 통해 모든 상자에 같은 색깔의 공이 들어가도록 옮길 수 있다.

- 진출 차수가 1 이하인 정점을 아무거나 하나 고른다.
- 해당 정점을 빈 상자로 옮긴다.
- 이제 다음 과정을 한 사이클에 있는 정점을 모두 처리할 때까지 반복한다.
 - 방금 선택하여 이동한 정점을 u 라고 하자.
 - 모든 정점 v 에 대해서 간선 (v, u) 가 존재하면 제거한다.
 - G 에서 진출 차수가 0이고, G' 에서 차수가 1 이하인 정점 w 를 아무거나 하나 고른다.
 - 정점 w 의 진출 차수가 0이라는 것은 색깔이 w 인 공이 모두 상자의 제일 위에 있다는 의미이다. 또한, 차수가 1 이하인 정점이라는 것은 한번의 이동으로 해당 색깔의 공을 한 상자로 모을 수 있다는 의미이다. 따라서, 색깔이 w 인 두 공 중 하나를 선택하여 다른 상자로 옮기면 한 상자로 공을 모을 수 있다.

그러나 한 사이클에서 진출 차수가 2인 정점이 2개 이상 있으면 위 과정을 끝까지 진행할 수 없다. 그렇지 않은 경우, 한 사이클에 포함된 공을 각 상자에 같은 색깔의 공이 들어가도록 옮기는 횟수는 다음과 같다.

- 한 사이클에 포함된 정점의 수를 V 라고 하자.

- $V = 1$ 이면, 0번
- $V > 1$ 이면, $V + 1$ 번

모든 사이클에 대해 위 횃수를 더하면 정답을 구할 수 있다.

“초등부 4번. 양손에 V” 문제 풀이

작성자: 조승현

부분문제 1

두 번의 V자 색칠하기를 하는 모든 경우를 다 시도해 볼 수 있다. 흰 격자를 선택하는 방법은 $O(NM)$ 가지이고, V자 색칠하기를 시뮬레이션 하는 시간은 $O(\min(N, M))$ 이므로 첫 V자 색칠하기, 두번째 V자 색칠하기, 시뮬레이션까지 고려한 시간복잡도는 $O(NM \cdot NM \cdot N) = O(N^2 M^2 \min(N, M))$ 이다.

부분문제 2

i 행 j 열의 격자를 편의상 (i, j) 로 표기하기로 한다.

(i, j) 에서 V자 색칠하기를 했을 때 색칠되는 격자의 수를 $C[i][j]$ 라 하면, (i, j) 가 흰색일 때 $C[i][j]$ 는 $((i, j)$ 부터 왼쪽 위 대각선으로 연속된 흰색 격자의 수) + $((i, j)$ 부터 오른쪽 위 대각선으로 연속된 흰색 격자의 수) - 1이 된다.

- $L[i][j]$: (i, j) 부터 왼쪽 위 대각선으로 연속된 흰색 격자의 수
- $R[i][j]$: (i, j) 부터 오른쪽 위 대각선으로 연속된 흰색 격자의 수

(i, j) 가 흰색인 경우 $L[i][j] = L[i-1][j-1] + 1$, 아닌 경우 $L[i][j] = 0$ 이며, $R[i][j]$ 도 유사하게 계산할 수 있다. 따라서, 모든 $L[i][j]$ 및 $R[i][j]$ 값을 계산할 수 있고, $C[i][j]$ 테이블 역시 $O(NM)$ 시간복잡도로 채울 수 있다.

첫 V자 색칠하기를 하는 각각의 경우에 대해, 첫 V자 색칠하기 후 상태에서 모든 $C[i][j]$ 를 $O(NM)$ 시간에 구해 놓으면 문제를 해결할 수 있다. 이 때 총 시간복잡도는 $O(N^2 M^2)$ 이다.

부분문제 4

$i + j$ 가 짝수인 (i, j) 를 짝수 칸, $i + j$ 가 홀수인 (i, j) 를 홀수 칸이라 하자. V자 색칠하기를 하는 두 칸의 기우성에 따라 경우의 수를 나눌 수 있다. 먼저, 짝수 칸 하나와 홀수 칸 하나에서 했다면 두 V자 색칠하기는 서로에게 영향을 끼치지 않는다. 즉, 초기 상태에서 $C[i][j]$ 를 계산해 놓고, 짝수 칸의 최댓값과 홀수 칸의 최댓값의 합을 구하면 이 케이스에서 최적이다.

첫 번째 V자 색칠하기에서 색칠된 칸들의 집합을 S_1 두 번째 V자 색칠하기로 색칠된 칸들의 집합을 S_2 라 할 때, 다음 셋 중 하나가 성립한다:

1. 정수 k 가 존재하여 S_1 의 모든 격자 (i, j) 는 $i + j \leq k$, S_2 의 모든 격자 (i, j) 는 $i + j > k$ 가 성립한다. (또는 그 반대)
2. 정수 k 가 존재하여 S_1 의 모든 격자 (i, j) 는 $i - j \leq k$, S_2 의 모든 격자 (i, j) 는 $i - j > k$ 가 성립한다. (또는 그 반대)
3. 선택한 두 격자에 대해 둘 중 위쪽인 격자를 중심으로 대각선을 그어 격자판을 4개의 영역으로 나누면 다른 격자는 아래 영역에 포함되며, 이 때 두 V자 색칠은 서로 영향을 끼치지 않는다.

1번 또는 2번 케이스는 두 V자가 서로 $i + j = k$ 나 $i - j = k$ 직선 기준으로 나누어진 두 영역에 각각 포함됨을 뜻하며, 3번 케이스는 두 V자 중 하나는 위쪽, 하나는 아래쪽 V자가 되어 서로 독립적인 경우이다.

1, 2번 케이스는 다음에 정의되는 두 테이블 DL, DR 의 값을 계산하여 해결할 수 있다.

- $DL[i][j] : (i, j)$ 에서 출발하여 왼쪽 아래 대각선으로 가다가 왼쪽 위 대각선으로 가는 흰색 칸으로만 이루어진 경로의 최대 칸 수
- $DR[i][j] : (i, j)$ 에서 출발하여 오른쪽 아래 대각선으로 가다가 오른쪽 위 대각선으로 가는 흰색 칸으로만 이루어진 경로의 최대 칸 수

DL 테이블은 $DL[i][j] = \max(DL[i+1][j-1] + 1, L[i][j])$ 의 점화식으로 $O(NM)$ 시간에 계산할 수 있고, DR 도 동일한 방법으로 가능하다. 그리고 DL, DR, C 테이블을 이용하면 $i + j \leq k$ 인 칸들만 사용하는 V자의 최대 길이, $i + j > k$ 인 칸들만 사용하는 V자의 최대 길이 등을 빠르게 계산할 수 있다. 모든 k 에 대해 $i + j \leq k$ 인 칸에서의 V자, $i + j > k$ 에서의 V자, $i - j \leq k$ 인 칸에서의 V자, $i - j > k$ 에서의 V자의 최대 길이를 모두 구하면 충분하고, 따라서 이 케이스는 $O(NM)$ 시간에 처리 가능하다.

3번 케이스는 $M[i][j] = \max(C[i][j], M[i-1][j-1], M[i-1][j], M[i-1][j+1])$ 로 정의되는 DP 테이블 M 를 채우면 두 V자 중 아래가 (i, j) 를 선택한 것일 때 위쪽 V자는 $M[i-1][j]$ 칸만큼 색칠하는 것이 최적임을 알 수 있다(직사각형에서 prefix max를 사용한 것과 동일하다). 따라서, $O(NM)$ 시간에 처리 가능하다.

종합하여, 다이나믹 프로그래밍으로 $O(NM)$ 시간에 문제를 해결할 수 있다. 구현을 효율적으로 하지 못하여 $O(N^2M)$ 이나 $O(NM^2)$ 로 문제를 해결하는 코드를 작성하면 부분문제 3까지만 점수를 받을 수 있다.